Loop Engineering is just K8s-style Cybernetics

One control loop in Watt's governor, Kubernetes and coding agents

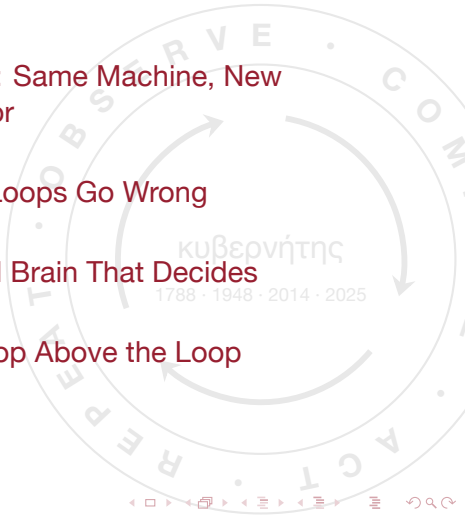
Sayantan Das

MLOps North

November 5, 2026



1. The Steersman
2. Cybernetics in Five Minutes
3. Kubernetes: Cybernetics for Infrastructure
4. Agents: Same Machine, New Actuator
5. When Loops Go Wrong
6. A Small Brain That Decides
7. The Loop Above the Loop



1 The Steersman

2 Cybernetics in Five Minutes

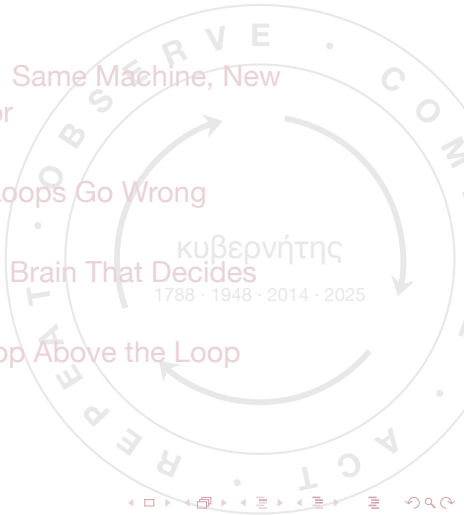
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The title says the same thing twice

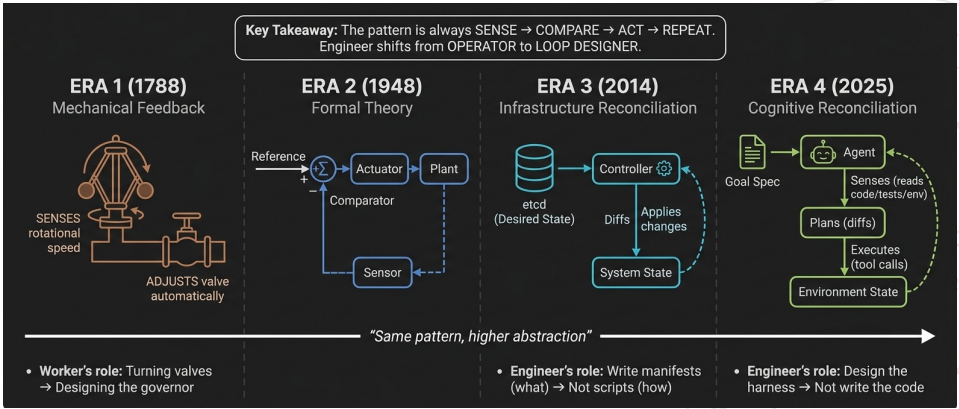
- κυβερνήτης (*kybernētēs*): the steersman of a ship
- **1948**: Norbert Wiener names the science of feedback and control cybernetics after this word
- **2014**: Google releases a container orchestrator as open source and names it Kubernetes, Greek for helmsman
- **2025**: engineers put a language model in the loop and call the practice loop engineering



Claim

Loop engineering uses the machine from Watt's governor and Kubernetes at a higher level: **state a goal, sense, compare, act, repeat.**

Four eras, one pattern



In each era, the human moves up one level: the worker who turned the valve now **designs the governor**.

1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

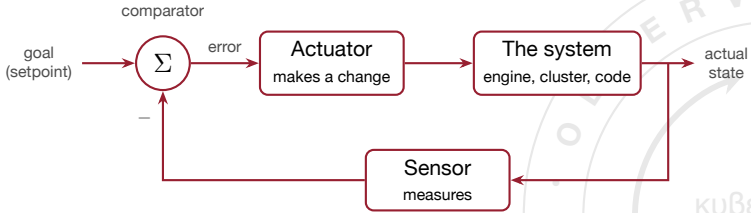
5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



Every feedback loop has the same four parts



	Goal	Sensor	Comparator	Actuator
Watt governor (1788)	target speed	flyball weights	linkage	steam valve
Thermostat	21 °C	thermometer	switch	furnace
Kubernetes (2014)	YAML manifest	API server watch	controller diff	controller writes
Agent loop (2025)	task spec + checks	tests, logs, browser	diff vs. goal	LLM edits code

How control loops fail: Maxwell, *On Governors* (1868)

Steam-engine governors hunted: they overshot the target speed, corrected, and overshot again. Maxwell explained the cause in 1868, and his paper is one of the first texts of control theory.

Control term	What it means	In an agent loop
Negative feedback	push against the error	undo a change that made the result worse
Damping	slow down after an overshoot	make smaller changes, and wait longer after a miss
Oscillation (hunting)	overshoot, correct, repeat	fix one test, break two, and repeat
Steady-state error	stop short of the goal	the agent cannot close the last gap
Saturation limits	the valve has a maximum opening	limits on rounds, time and money

Ashby's Law: you can only control what you can sense

Law of Requisite Variety (W. Ross Ashby, *An Introduction to Cybernetics*, 1956)

Only variety can absorb variety. A regulator can correct only the disturbances that it can identify.

- In an agent loop, **the checks are the sensors**. The loop cannot be better than its checks.
- The loop ships any bug that the tests do not detect.
- Goodhart's law applies: a measure that becomes a target stops being a good measure.
- For an agent, the cheapest way to make a failing test pass is to edit the test.

Add sensors before you add a smarter planner.

1 The Steersman

2 Cybernetics in Five Minutes

3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The reconciliation loop

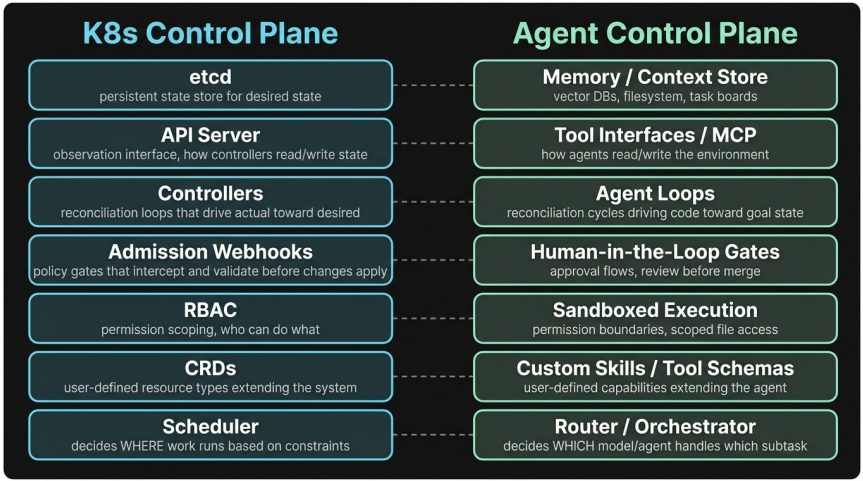
1. **Desired state**: a manifest says what should exist
2. **Observe**: read the current state
3. **Diff**: compare desired vs. actual
4. **Act**: make the smallest change that closes the gap
5. **Repeat**: forever

Level-triggered, not edge-triggered

A controller acts on *the current state*, not on the last event. If it misses an event, the next pass still finds the gap.

The controller does not run a script from start to end. It calculates only the *next correction*. If a pod stops, the loop starts a new pod, so self-healing comes from the loop, not from special-case code.

The control plane, mapped



MLOps already runs on these loops

- **GitOps** (Argo CD, Flux): git holds the desired state, and a controller syncs the cluster to it.
- **Continuous training**: if a metric drops below the line, retrain until the metric is above the line again.
- **Progressive delivery**: if canary metrics get worse, the system rolls back.
- **Data validation before training**: a gate rejects bad input, like an admission webhook.

What is new

Only the actuator. A language model now makes the change, and it can make *any* change, bad changes too. The other parts stay the same.

1 The Steersman

2 Cybernetics in Five Minutes

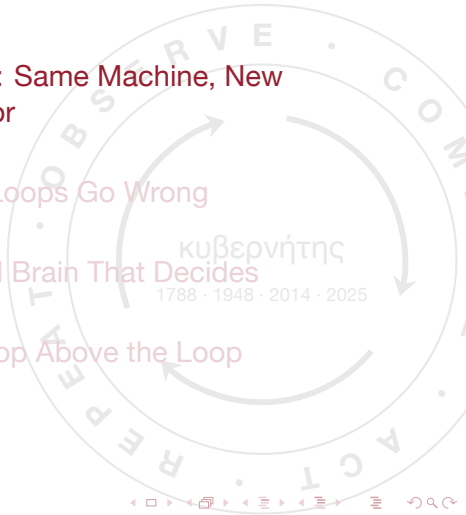
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

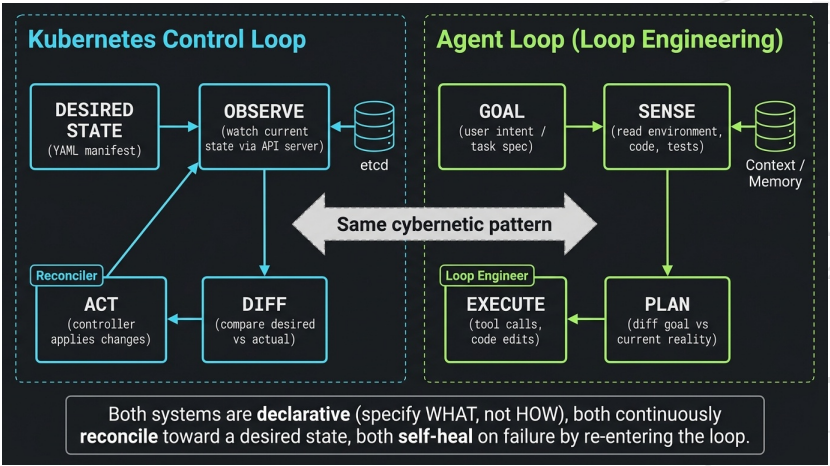
5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



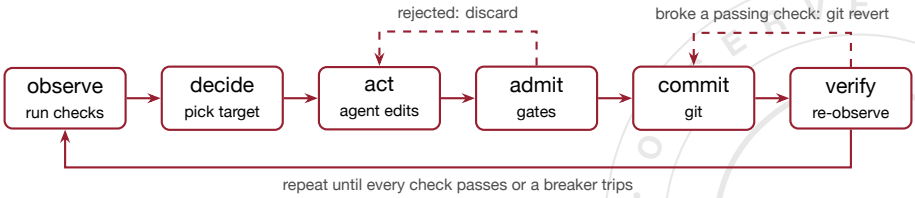
Same loop, new actuator



Design rules that transfer directly

Rule	In Kubernetes	In an agent loop
Declarative	the manifest says *what*, not *how*	“all checks pass” is better than a 47-step checklist
Observe first	read the API server before each action	read the code and run the tests before each edit
Small diffs	change one thing per pass	one target per round, with a limit on diff size
Idempotent	a second pass causes no damage	“the file contains X”, not “append Y”
Crash-only	a controller can stop at any time	git stores the progress, and the agent stores nothing
Bounded ownership	labels and selectors	one loop owns one directory

loopctl in one picture



- loopctl has about 970 lines of Python and uses only the standard library. Git is its etcd: each kept change is a commit.
- The agent is a shell command: pi, Claude Code, Codex or a script.
- You can stop loopctl at any time. When you start it again, it continues from the last commit with the same budgets.

The goal is a file

```
name  = "churn"
goal  = "Ship-ready churn model: the data contract holds, holdout AUC is at least 0.85,
     and the model card documents the model."
actor = 'pi -p "$(cat)"'      # any agent CLI; the prompt arrives on stdin
fresh_clone = true            # observe a clean clone every round: no stale reads

[[checks]]                    # desired state = every check exits 0
name = "quality"               # (plus "schema" and "model-card")
run  = "python3 eval.py --min-auc 0.85"

[[admission]]                 # policy gate: hands off the sensors
name = "sensors-untouched"
run  = "git diff --cached --quiet -- eval.py validate.py check_card.py schema.json data
    /"

[limits]                      # the safety governor
max_diff_lines   = 80         # damping: big changes are rejected
stall_iterations = 3          # steady-state error: give up
max_regressions  = 2          # oscillation: stop thrashing
```

Every part has a cybernetic name

loopctl	Cybernetics	Kubernetes
`goal` + `[[checks]]`	setpoint	manifest
the check run	sensor	watch via API server
pass/fail comparison	comparator	diff
the agent (`actor`)	actuator	controller writes
`git revert` on regression	negative feedback	rollback
backoff, `max_diff_lines`	damping	rate-limited requeue
`max_regressions`	oscillation guard	none
`stall_iterations`	steady-state error	none
`[[admission]]`, approval	none	admission webhooks
lease file, sandbox	none	RBAC, namespaces
`[decider]`	controller logic	scheduler

Demo 1: a churn model reaches its setpoint (offline)

```
$ ./examples/churn/demo.sh
[churn] observe: 0/3 passing, failing ['schema', 'quality', 'model-card']
[churn] act #1: target `schema`
[churn] committed a0df3d90 (5 lines)
[churn] observe: 1/3 passing, failing ['quality', 'model-card']
[churn] act #2: target `quality`
[churn] committed 37e494e0 (3 lines)
[churn] observe: 1/3 passing, failing ['quality', 'model-card']
[churn] backoff 0.2s
[churn] act #3: target `quality`
[churn] committed f7532e1e (2 lines)
[churn] observe: 2/3 passing, failing ['model-card']
[churn] act #4: target `model-card`
[churn] committed 84efe5aa (22 lines)
[churn] observe: 3/3 passing
[churn] converged
```

The model has three planted bugs: `contract` has no one-hot encoding, training and serving use different scaling, and `support_calls` is missing. Round 2 increases AUC from 0.75 to 0.82, and loopctl keeps that change, but AUC is still below 0.85. Round 3 increases AUC to 0.874.

Demo 2: a real agent as the actuator

```
1 actor = 'pi -p "$(cat)"'
```

```
$ ./examples/churn/demo.sh loop.pi.toml
[churn-pi] act #1: target `schema`
[churn-pi] committed de9e9ea8 (5 lines)
[churn-pi] act #2: target `quality`
[churn-pi] committed 9755f658 (5 lines)
[churn-pi] act #3: target `model-card`
[churn-pi] committed 9accb5a8 (28 lines)
[churn-pi] converged
churn-pi: 3 actions, 3 commits, 0 rejected,
  0 reverted, 205.9s acting
```

pi found these fixes from the AUC value only:

```
-NUMERIC = ["tenure_months", "monthly_charges"]
+NUMERIC = ["tenure_months", "monthly_charges",
+           "support_calls"]
-    return sigmoid(sum(w * v for w, v in zip(...x)) + b)
+    xs = scale(x, model["scaler"])
+    return sigmoid(sum(w * v for w, v in zip(...xs)) + b)
```

pi fixed both quality bugs (the missing feature and the scaling skew) in one 5-line change. Holdout AUC is 0.874, and no gate rejected the change.

Demo 3: brain outside, hands inside

loopctl runs each check in a temporary Apple container VM:

```
container run --rm --network none \
  -v "$PWD":/work:ro -w /work \
  python:3.12-slim "$@"
```

```
1 run = "./incvm python3 -m unittest ..."
```

Code from the agent sees this inside the VM:

```
home visible: False
network: blocked URLError
write /work: blocked Read-only file system
```

loopctl and the agent stay on the host, so the agent cannot change the grading. Each VM takes about 1 s, and the full loop converged in 55 s.

1 The Steersman

2 Cybernetics in Five Minutes

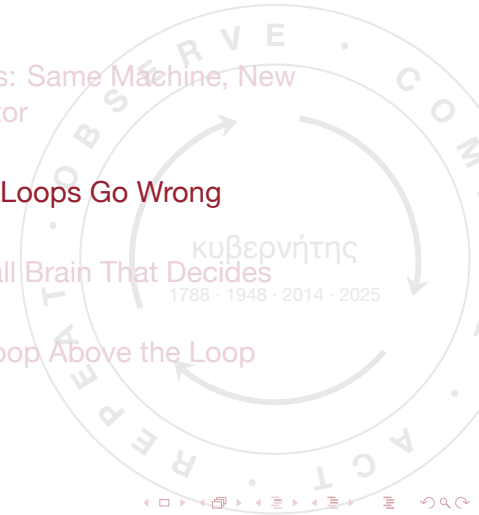
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



Kubernetes already learned these lessons

Failure	Looks like	loopctl's answer
Thrashing	fix one test, break two	verify each commit, revert a regression, stop after `max_regressions`
Stale reads	decisions from old data	run the checks again each round, on a fresh clone if you want
Endless loop	the goal is out of reach	limits on rounds and time, and `stall_iterations`
Fighting loops	two loops, one directory	a lease file, and a halt if the agent edits outside the directory
Sensor tampering	the agent edits the test	admission checks reject the change
Blind loop	no signal to steer by	the loop cannot fix this: add checks

Live failure: loopctl stops the thrashing

A worker that fixes `add` but breaks `mul` every time:

```
[calc] observe: 1/4 passing, failing ['add', 'sub', 'div']
[calc] act #1: target `add`
[calc] committed 84dcb427 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 84dcb427: broke ['mul']
[calc] act #2: target `add`
[calc] committed 8a047e50 (4 lines)
[calc] observe: 1/4 passing, failing ['sub', 'mul', 'div']
[calc] reverted 8a047e50: broke ['mul']
[calc] thrashing: 2 reverted regressions
```

loopctl reverts each regression (negative feedback) and stops after two, so the loop does not hunt. Git keeps every attempt for a human to read.

Live failure: the agent edits the check

This worker tries to fix `quality` with a lower AUC bar in `eval.py`:

```
[churn] act #2: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] act #3: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] act #4: target `quality`
[churn] rejected: admission[sensors-untouched]: exit 1
[churn] unreachable: no progress in 3 iterations; still failing ['quality', 'model-card']
```

- One line of shell in an admission webhook blocks this Goodhart attack.
- A second gate blocks model code that reads the holdout set (`no-holdout-in-model-code`).
- The loop stops and does not ship a false pass.

1 The Steersman

2 Cybernetics in Five Minutes

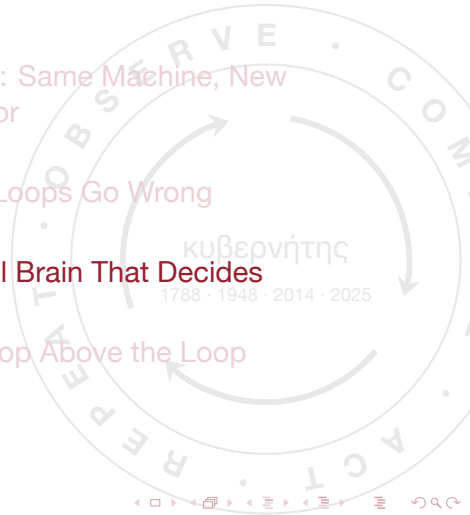
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The scheduler picks a node, and the kubelet runs the pod

Kubernetes scheduler

- filter: which nodes *can* run this pod?
- score: which is best?
- pick: one node gets the pod

Same split for agents

- a small, fast model **decides**
- a big, slow model **does**
- the checks **judge**

TypeSafe Jev

Jev answers typed questions with probabilities, not free text. loopctl sends one request per round:

- move: act / ask a human / stop
- target: which failing check next

Guardrails: the decider cannot judge pass or fail, and it cannot pick a passing check. Under 50% confidence, it asks a human. If it fails, loopctl uses the simple rule.

Rules vs. Jev: an impossible check listed first

`gpu` runs `nvidia-smi`: no code change can make it pass on a Mac.

Built-in rule (first failing check)

```
act #1: target `gpu`
rejected: no_op: actor made no changes
act #2: target `gpu`
rejected: no_op: actor made no changes
act #3: target `gpu`
rejected: no_op: actor made no changes
unreachable: no progress in 3 iterations
```

0 of 3 fixable checks fixed.

Jev decider

```
decide: act `add` (0.98)
  target add=0.67 div=0.22 sub=0.09 gpu=0.02
committed 22ef9645
decide: act `sub` (0.98)
  target sub=0.90 div=0.10 gpu=0.00
committed 6f7b3f5c
decide: act `div` (0.96)
  target div=1.00 gpu=0.00
committed 3d5f5a34
decide: stop `gpu` (0.97)
  stop=0.99 ask_human=0.01 act=0.00
unreachable: decider chose stop
```

Jev fixed **3 of 3**, then stopped. The run took about 3 s.

1 The Steersman

2 Cybernetics in Five Minutes

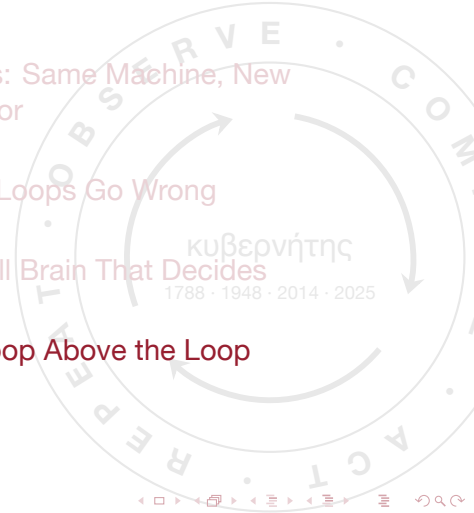
3 Kubernetes: Cybernetics for Infrastructure

4 Agents: Same Machine, New Actuator

5 When Loops Go Wrong

6 A Small Brain That Decides

7 The Loop Above the Loop



The hill-climbing machine

The same loop, one level up:

1. ship a v0 agent
2. record what it does (traces)
3. mine traces for failure patterns
4. turn failures into checks
5. improve the agent, then repeat

Here the setpoint is *agent quality*, and you are the actuator.

```
$ loopctl report loop.jev.toml
calc-jev: 3 actions, 3 commits, 0 rejected,
  0 reverted, 0.1s acting
last outcome: unreachable (decider chose stop)

checks observed failing:
     4  gpu
     3  div
     2  sub
     1  add

decisions:
     3  decider: act
     1  decider: stop
```

Try it yourself: the churn task

```
git clone https://github.com/ucalyptus/loop-engineering
cd loop-engineering
./examples/churn/demo.sh
./examples/churn/demo.sh loop.pi.toml
```

The first command runs offline in about 5 seconds. The second command uses your agent. You need Python 3.11+ and git, and you do not need `pip`. The repo is private now, so ask me for access.

1. Watch it converge. Why did `quality` take two rounds?
2. Use your own agent: pi, Claude Code or Codex.
3. Be the adversary: cheat, then find a cheat the gates miss.
4. Add the sensor that the loop does not have: recall, calibration or AUC per segment.
5. Stretch: let Jev decide.

Takeaways

1. **Declare the exit condition, not the path.** Checks are your setpoint.
2. **Your checks are your sensors.** The loop is only as good as what it can see.
3. **Never let the actuator touch the sensor.** Brain outside, hands inside.
4. **Observe before every action.** Do not trust the memory of the agent.
5. **Make one small change per round, and verify it.** Undo any change that makes the result worse.
6. **Keep actions safe to repeat, and keep state outside the agent.** Then you can stop the agent at any time.
7. **Install a governor.** Set limits on rounds, time and money, and add a human gate.
8. **Let a small model decide and a big model do.** The checks still judge.

Loop engineering is Kubernetes for agents, and Watt's governor for software.

Questions?

Questions?

Your checks are the goal and the sensors. Keep the judge outside the loop.

Contact

Thanks!

Sayantan Das
linkedin.com/in/ucalyptus
github.com/ucalyptus
blog.ucalyptus.me

Talk to me after the session about:

- setup, the churn task, or your own agent
- access to the private `loop-engineering` repo

The blog post this talk is based on

Further reading

- S. Das, *Loop Engineering Is Just K8s-Style Orchestration* (blog.ucalyptus.me)
- N. Wiener, *Cybernetics: Or Control and Communication in the Animal and the Machine*, 1948
- W. R. Ashby, *An Introduction to Cybernetics*, 1956
- J. C. Maxwell, *On Governors*, Proceedings of the Royal Society, 1868
- controller-runtime and the Kubernetes scheduler docs
- Apple container and pi-cvm (github.com/ucalyptus/pi-cvm)
- TypeSafe docs (docs.typesafe.ai)